GRPO (Group Relative Policy Optimization) 프로세스 분석

1. GRPO 개요

GRPO(Group Relative Policy Optimization)는 Open R1 프로젝트의 핵심 강화학습 알고리즘으로, 기존의 PPO(Proximal Policy Optimization)를 개선한 방법입니다. 여러 응답을 그룹으로 묶어 상대적 품질을 비교하여 정책을 최적화합니다.

1.1 GRPO vs PPO 차이점

| 특징 | PPO | GRPO |

|------|-----|------|

| **비교 방식** | 절대적 보상 기준 | 그룹 내 상대적 비교 |

| **안정성** | 보상 스케일에 민감 | 보상 스케일에 강건 |

| **효율성** | 개별 응답 최적화 | 그룹 단위 최적화 |

| **수렴성** | 불안정할 수 있음 | 더 안정적 수렴 |

1.2 핵심 아이디어

```
# PPO: 절대적 보상으로 정책 업데이트

reward = compute_reward(response)

loss = -log_prob * advantage


# GRPO: 그룹 내 상대적 순위로 정책 업데이트
```

```
group_rewards = [compute_reward(r) for r in group_responses]

relative_advantages = compute_relative_advantages(group_rewards)

loss = -sum(log_prob * relative_advantage for log_prob, relative_advantage in  z
```

2. 핵심 구현 (`src/open_r1/grpo.py`)

2.1 메인 함수 구조

```
def  main(script_args, training_args, model_args):

# 1. 환경 설정

set_seed(training_args.seed)

setup_logging()

# 2. 데이터, 토크나이저, 모델 로드

dataset = get_dataset(script_args)

tokenizer = get_tokenizer(model_args, training_args)

model = get_model(model_args, training_args)

# 3. 보상 함수 설정

reward_funcs = get_reward_funcs(script_args)

# 4. 대화 형식 변환

def  make_conversation(example):

prompt = []

if training_args.system_prompt:

prompt.append({"role": "system", "content": training_args.system_prompt})
```

```
prompt.append({"role": "user", "content": example[script_args.dataset_prompt_col

return {"prompt": prompt}

# 5. GRPO 트레이너 초기화 및 훈련

trainer = GRPOTrainer(

model=model,

tokenizer=tokenizer,

args=training_args,

train_dataset=dataset,

reward_funcs=reward_funcs,

...

)

trainer.train()
```

2.2 보상 함수 시스템

다중 보상 함수 지원

```
# 보상 함수 등록

reward_funcs = get_reward_funcs(script_args)


# 예시: 수학 문제용 보상 함수

reward_funcs = {

"accuracy": accuracy_reward, # 정답 여부

"format": format_reward, # 형식 준수
```

```
"tag_count": tag_count_reward # 태그 개수
}
```

보상 함수 가중치

```yaml
# YAML 설정에서 보상 함수 가중치 조정
reward_funcs:
accuracy:
weight: 0.7
format:
weight: 0.2
tag_count:
weight: 0.1
```

2.3 vLLM 백엔드 통합

단일 노드 설정 (Colocate 모드)

```bash
# vLLM을 훈련 프로세스와 같은 노드에서 실행
ACCELERATE_LOG_LEVEL=info \
accelerate launch --config_file recipes/accelerate_configs/zero3.yaml \
src/open_r1/grpo.py --config recipes/DeepSeek-R1-Distill-Qwen-1.5B/grpo/config
--vllm_mode colocate
```

멀티 노드 설정 (분리 모드)

```
# 1단계: vLLM 서버 시작 (별도 노드)
CUDA_VISIBLE_DEVICES=0 trl vllm-serve --model Qwen/Qwen2.5-1.5B-Instruct


# 2단계: GRPO 훈련 (훈련 노드들)
CUDA_VISIBLE_DEVICES=1,2,3,4,5,6,7 ACCELERATE_LOG_LEVEL=info \
accelerate launch --config_file recipes/accelerate_configs/zero2.yaml --num_
src/open_r1/grpo.py --config recipes/Qwen2.5-1.5B-Instruct/grpo/config_demo.ya
```

3. 훈련 설정 및 최적화

3.1 GRPO 하이퍼파라미터

```yaml
# 예시 GRPO 설정
learning_rate: 3.0e-6
num_train_epochs: 1
max_seq_length: 32768
per_device_train_batch_size: 1
gradient_accumulation_steps: 8
num_sample_generations: 4  # 그룹당 생성할 응답 수
response_length: 2048  # 최대 응답 길이
temperature: 0.7  # 생성 온도
```

```
top_p: 0.9  # 핵 샘플링
kl_coeff: 0.1  # KL 발산 계수
```

3.2 채팅 템플릿 주의사항

DeepSeek 증류 모델의 경우 특별한 채팅 템플릿 처리가 필요합니다:

```
# config_demo.yaml 예시
chat_template: |
{% for message in messages %}
{% if message['role'] == 'user' %}
<|im_start|>user
{{ message['content'] }}<|im_end|>
<|im_start|>assistant
<think>
{% elif message['role'] == 'assistant' %}
{{ message['content'] }}
{% endif %}
{% endfor %}
```

주의사항:

- DeepSeek 모델은 `<think>` 태그 내용을 채팅 템플릿에서 제외
- 어시스턴트 응답에 `<think>` 프리필 적용
- 형식 보상 함수와의 충돌 방지 필요

4. 코드 실행 보상 시스템

4.1 지원 샌드박스 제공자

E2B (Python 특화)

```
# .env 파일 설정
E2B_API_KEY="e2b_xxx"

# 의존성 설치
uv pip install -e '.[code]'

# 설정 예시
provider_type: e2b
e2b_router_url: 1.2.3.4:8000 # 선택적 라우터 사용
```

Morph (다중 언어 지원)

```
# 설치 및 설정
pip install morphcloud
echo 'MORPH_API_KEY="YOUR_KEY"' >> .env

# 설정 예시
provider_type: morph
```

```
morph_router_url:  1.2.3.4:8000
```

4.2 데이터셋 요구사항

코드 실행 보상을 위한 데이터셋은 `verification_info` 컬럼이 필요합니다:

```
{
"language": "python",
"test_cases": [
{
"input": "4\n4\n0001\n1000\n0011\n0111\n",
"output": "1\n3\n",
"type": "stdin_stdout"
}
]
}
```

4.3 경쟁 프로그래밍 지원

IOI (International Olympiad in Informatics)

```
# IOI 문제용 설정
reward_func: ioi_code_reward
ioi_provider: "morph"  # 또는 "piston"
dataset_name: open-r1/ioi
```

CodeForces

```
# 테스트 케이스 다운로드
huggingface-cli download open-r1/codeforces \
--repo-type=dataset \
--include='generated_tests/*.parquet' \
--max-workers=8 \
--local-dir PATH_TO_SAVE_TESTCASES

# 환경 변수 설정
echo 'CF_TESTS_FOLDER=PATH_TO_SAVE_TESTCASES' >> .env

# CodeForces 설정
reward_func: cf_code_reward
dataset_name: open-r1/codeforces
```

5. 라우터 서비스

5.1 E2B 라우터

```
# CPU 노드에서 라우터 시작
sbatch slurm/e2b_router.slurm
```

```
# 훈련 설정에서 라우터 URL 지정
e2b_router_url: 1.2.3.4:8000
```

5.2 Morph 라우터

```
# CPU 노드에서 라우터 시작
sbatch slurm/morph_router.slurm

# 훈련 설정에서 라우터 URL 지정
morph_router_url: 1.2.3.4:8000
```

5.3 라우터의 장점

- **속도 제한 관리**: API 호출 속도 제한 자동 처리
- **부하 분산**: 여러 훈련 작업 간 요청 분산
- **안정성**: 네트워크 오류 시 자동 재시도
- **모니터링**: 실행 통계 및 오류 로깅

6. 실제 사용 예시

6.1 기본 GRPO 훈련

```
# 1.5B 모델로 데모 훈련
ACCELERATE_LOG_LEVEL=info accelerate launch \
--config_file recipes/accelerate_configs/zero2.yaml \
```

```
--num_processes=7 \
src/open_r1/grpo.py \
--config  recipes/Qwen2.5-1.5B-Instruct/grpo/config_demo.yaml
```

6.2 코드 실행 보상 훈련

```
# vLLM 서버 시작
CUDA_VISIBLE_DEVICES=0  trl  vllm-serve  --model  Qwen/Qwen2.5-1.5B-Instruct

# 코드 실행 보상으로 훈련
CUDA_VISIBLE_DEVICES=1,2,3,4,5,6,7  ACCELERATE_LOG_LEVEL=info  \
accelerate  launch  --config_file  recipes/accelerate_configs/zero2.yaml  --num_
src/open_r1/grpo.py  --config  recipes/Qwen2.5-1.5B-Instruct/grpo/config_demo_co
```

6.3 IOI 문제 훈련

```
ACCELERATE_LOG_LEVEL=info  accelerate  launch \
--config_file recipes/accelerate_configs/zero2.yaml \
--num_processes=7 \
src/open_r1/grpo.py \
--config  recipes/Qwen2.5-1.5B-Instruct/grpo/config_demo_code_ioi.yaml
```

6.4 CodeForces 훈련

```
sbatch  --job-name=cf-grpo  --nodes=2  slurm/train.slurm \
--model Qwen2.5-Coder-7B-Instruct \
--task  grpo \
--config codeforces \
--accelerator  zero3 \
--dp 8 --tp 1
```

7. Slurm 클러스터 실행

7.1 기본 Slurm 명령

```
# 단일 노드 GRPO (colocate 모드)
sbatch  --job-name=grpo-demo  --nodes=1  slurm/train.slurm \
--model Qwen2.5-1.5B-Instruct \
--task  grpo \
--config demo \
--accelerator  zero2

# 멀티 노드 GRPO (1+N 노드: 1개 vLLM 서버 + N개 훈련 노드)
sbatch  --job-name=grpo-multi  --nodes=2  slurm/train.slurm \
--model Qwen2.5-1.5B-Instruct \
--task  grpo \
--config demo \
```

```
--accelerator zero2 \
--dp 4 --tp 2
```

7.2 병렬화 전략

데이터 병렬 (DP)

```
--dp 8 # 8개 GPU에 데이터 병렬 분산
```

텐서 병렬 (TP)

```
--tp 2 # 2개 GPU에 모델 텐서 분산
```

혼합 병렬

```
--dp 4 --tp 2 # 4-way 데이터 병렬 × 2-way 텐서 병렬
```

8. 성능 최적화

8.1 메모리 최적화

```
# DeepSpeed ZeRO-2 설정 (GRPO 권장)
deepspeed_config:
zero_stage: 2
offload_optimizer_device: cpu
```

```
gradient_accumulation_steps: 8

train_micro_batch_size_per_gpu: 1
```

8.2 생성 최적화

```
# 효율적인 생성 설정

num_sample_generations: 4  # 그룹 크기

response_length: 2048  # 응답 길이 제한

temperature: 0.7  # 적절한 다양성

top_p: 0.9  # 핵 샘플링
```

8.3 보상 계산 최적화

```
# 비동기 보상 계산

async  def  compute_rewards_async(completions, reward_funcs):

tasks = []

for func_name, func in reward_funcs.items():

task = asyncio.create_task(func(completions))

tasks.append(task)

results = await asyncio.gather(*tasks)

return dict(zip(reward_funcs.keys(), results))
```

9. 문제 해결

9.1 일반적인 오류

vLLM 연결 오류

```
# 해결 방법
export VLLM_WORKER_MULTIPROC_METHOD=spawn
# vLLM 서버가 올바른 포트에서 실행 중인지 확인
curl http://localhost:8000/v1/models
```

메모리 부족

```
# 배치 크기 감소
per_device_train_batch_size: 1
gradient_accumulation_steps: 16

# 응답 길이 제한
response_length: 1024
max_seq_length: 16384
```

보상 함수 오류

```
# 보상 함수에서 None 반환 시 해당 예시 스킵
def safe_reward_func(completions, **kwargs):
try:
return compute_reward(completions)
```

```
except Exception as e:
logger.warning(f"Reward computation failed: {e}")
return [None] * len(completions)
```

9.2 성능 튜닝

느린 훈련 속도

```
# 해결 방법
use_liger_kernel: true  # 최적화된 커널
bf16: true  # 혼합 정밀도
dataloader_num_workers: 4  # 데이터 로딩 병렬화
```

불안정한 훈련

```
# KL 발산 계수 조정
kl_coeff: 0.05  # 더 보수적인 정책 업데이트

# 학습률 감소
learning_rate: 1.0e-6

# 그래디언트 클리핑
max_grad_norm: 1.0
```

GRPO는 Open R1의 핵심 강화학습 알고리즘으로, 올바른 설정과 최적화를 통해 강력한 추론 능력을 가진 모델을 훈련할 수 있습니다.